

Luxair Interview Terms Glossary

Azok a szavak, kifejezések és témák, amelyekre rákérdeztél - rövid magyar magyarázatokkal és angol interjúmondatokkal.

Hogyan használd ezt a doksit?

Ez nem egy teljes tankönyv, hanem egy gyors visszakereshető glossary. A cél az, hogy interjú előtt gyorsan át tudd nézni azokat a fogalmakat, amelyeknél megálltunk és tisztáztuk a jelentést.

A legtöbb elemnél három dolgot kapsz: rövid jelentés, miért fontos production/integration környezetben, és egy angol interjúmondat.

Csoport	Fogalmak
Performance és scaling	p95/p99, throughput, 100M/month sizing, WebSocket vs HTTP, I/O, readiness probe, VM vs container
Architecture és integration	upstream/downstream, coupling, fan-out, SOA vs ESB, coarse-grained services, choreography, aggregator, webhook
Resilience	graceful degradation, circuit breaker, bulkhead, backpressure, concurrency limit, stale data
Data és consistency	ACID, local vs distributed transaction, saga, outbox, reconciliation, DB constraints, SQL optimization, locks/deadlocks
Security	nonce, replay protection, revocation, webhook security
Observability	RED vs USE, logs/metrics/traces kapcsolódó gondolkodás

1. Performance és scaling

p95 és p99 latency

Röviden: A p95 azt jelenti, hogy a requestek 95%-a ennél gyorsabb volt. A p99 azt jelenti, hogy a requestek 99%-a ennél gyorsabb volt.

Miért fontos: Az átlag latency el tudja rejteni a lassú outliereket. Productionben a lassú tail, főleg p95 és p99, gyakran fontosabb, mint az átlag.

Példa:

```
Average latency: 120 ms
p95 latency: 400 ms
p99 latency: 1800 ms
```

Jelentés: a requestek 95%-a 400 ms alatt ment le, 99%-a 1800 ms alatt.

Interview one-liner: Average latency can hide slow outliers, so I would also check p95 and p99 latency to understand the tail latency users actually experience.

Throughput

Röviden: Időegység alatt mennyi munkát dolgoz fel a rendszer. API-knál tipikusan requests per second.

Miért fontos: A latency azt mondja meg, mennyi ideig tart egy request. A throughput azt mondja meg, mennyi requestet tud a rendszer befejezni egy adott időablakban.

Példa:

```
Incoming traffic: 100 rps
Completed responses: 60 rps
```

Traffic 100 rps, de a tényleges throughput csak 60 rps.

Interview one-liner: Throughput tells me how many requests the system can complete per unit of time, while latency tells me how long one request takes.

100M requests/month értelmezése

Röviden: 100 millió request havonta kb. 39 request/sec átlagban, ha 30 nappal számolunk.

Miért fontos: Ez önmagában nem extrém nagy szám, de a peak traffic, request complexity, DB terhelés, downstream fan-out és retry storm sokkal fontosabb.

Példa:

```
100,000,000 / (30 * 24 * 60 * 60) ≈ 38.6 rps average
```

Ha peak 10x, akkor már kb. 386 rps csúcsban.

Interview one-liner: 100 million requests per month is around 40 rps on average, but I would size for peak traffic, request complexity, downstream limits and p95/p99 latency targets.

Capacity planning és load testing

Röviden: Hidegen lehet becsülni, de pontos gépigényt csak reális load testtel és méréssel lehet mondani.

Miért fontos: Ugyanaz a requestszám teljesen más gépet igényel, ha az egyik endpoint csak cache-ből ad JSON-t, a másik pedig DB transactiont, payment providert és több partner API-t hív.

- Becsülni lehet: average rps, peak rps, concurrency, instance count.
- Mélni kell: p95/p99 latency, CPU per request, memory, event loop lag, DB latency, pool saturation.
- Korrekt módszer: egv instance kapacitását mérni, majd safety margin és redundancy alanián skálázni

Interview one-liner: I can make an initial estimate from traffic volume, but I would validate production sizing with realistic load tests and measure how much one instance can handle safely.

WebSocket vs HTTP overhead

Röviden: WebSocketnél egy kezdeti HTTP Upgrade után egy long-lived connectionen mennek a kisebb message frame-ek.

Miért fontos: Sok kis real-time üzenetnél ez hatékonyabb lehet, mint ismétlődő HTTP requestek, de modern HTTP is tud connection reuse-t és multiplexinget.

- HTTP/1.1: keep-alive, egy TCP kapcsolat több requesthez újrahasználható.
- HTTP/2: multiplexing, több párhuzamos stream egy TCP/TLS kapcsolat fölött.
- HTTP/3: QUIC/UDP, kevesebb TCP head-of-line jellegű probléma.
- gRPC klasszikusan HTTP/2 + Protocol Buffers.

Interview one-liner: WebSocket is efficient for real-time bidirectional communication, but modern HTTP also reuses connections. The bigger difference is the communication model, not only TCP reuse.

I/O

Röviden: Input/Output. Backendben főleg azt jelenti, amikor az app külső erőforrásra vár, például DB-re, API-ra, file-ra, Redisre vagy queue-ra.

Miért fontos: Node.js erős I/O-bound feladatokra, mert amíg vár egy külső válaszra, addig más requesteket is tud kezelni. CPU-heavy szinkron munka viszont blokkolhatja az event loopot.

Példa:

```
I/O-bound: await db.query(...)
CPU-bound: nagy kép/video feldolgozás, komoly számítás, nagy JSON feldolgozás
```

Interview one-liner: Node.js is strong for I/O-bound workloads because it can handle other requests while waiting for databases, APIs or queues.

Inbound és outbound

Röviden: Inbound az, ami bejön a service-be. Outbound az, amikor a service hív kifelé valamilyen dependency-t.

Miért fontos: Integration debuggingnál külön kell látni, hogy a probléma a bejövő trafficben van-e, vagy egy kifelé menő DB/API/queue hívásban.

Példa:

```
Inbound: Partner API -> Your API
Outbound: Your API -> Payment provider / Oracle DB / CRM / Email service
```

Interview one-liner: Inbound traffic comes into my service, while outbound traffic leaves my service towards dependencies like databases, internal APIs, external providers or brokers.

Readiness probe

Röviden: Health check, ami megmondja, hogy az instance készen áll-e production trafficet fogadni.

Miért fontos: Lehet, hogy a process már fut, de még tölt configot, nyit DB poolt vagy warmupol. Ilyenkor nem jó, ha a load balancer már requesteket küld rá.

Példa:

```
/live -> él-e a process?
/ready -> kaphat-e trafficet?
```

Interview one-liner: Readiness tells the platform whether an instance is ready to receive traffic, while liveness tells whether the process is alive and should be restarted.

Miért nem elég egy sima /health?

Röviden: Mert a liveness és readiness két külön kérdés. Az egyik restart döntéshez kell, a másik routing/load balancing döntéshez.

Miért fontos: Ha egy endpoint túl laza, trafficet küldhetünk nem ready instance-re. Ha túl szigorú, a platform restartolhat egészséges processt egy temporary dependency hiba miatt.

- /live: minimális check, az app process él-e.
- /ready: az instance tud-e requesteket kiszolgálni.
- /health: lehet emberi/monitoring összefoglaló, de platform probe-nak gyakran jobb különválasztani

Interview one-liner: A single health endpoint can be misleading because liveness and readiness answer different operational questions.

VM vs container

Röviden: A VM teljes virtuális gépet futtat saját guest OS-sel. A container az appot és dependency-ket csomagolja, de a host OS kernelt használja.

Miért fontos: Container gyorsabban indul, könnyebb, jobban illik microservice/cloud skálázáshoz. VM erősebb OS-szintű izolációt ad.

Interview one-liner: A VM virtualizes a full machine including its own OS, while a container packages the app and dependencies but shares the host kernel.

2. Architecture és integration

Upstream és downstream

Röviden: Upstream az, aki téged hív vagy tőled kér valamit. Downstream az, akit te hívsz, hogy elvégezd a munkát.

Miért fontos: Ha egy API lassú, fontos tudni, hogy a bejövő caller okozza-e a terhelést, vagy egy downstream dependency, például DB/payment/CRM lassú.

Példa:

```
Frontend / Partner -> Your API -> Payment / DB / CRM
```

```
Frontend és Partner: upstream
```

```
Payment, DB, CRM: downstream
```

Interview one-liner: An upstream system is the caller or producer, while a downstream system is a dependency my service calls to complete its work.

Coupling

Röviden: Azt jelenti, hogy két rendszer mennyire függ egymástól.

Miért fontos: Loose coupling esetén a service-ek contractokon keresztül kommunikálnak és belül változhatnak egymás törése nélkül. Tight coupling esetén belső DB schema, workflow vagy implementation detail is átfolyik más service-ekbe.

- Loose coupling: stabil API/event contract, kevés belső tudás.
- Tight coupling: shared database, belső státuskódokra vagy workflow-ra építés, túl sok sync call.

Interview one-liner: Coupling describes how strongly systems depend on each other. I would keep it low with clear contracts, ownership boundaries and asynchronous messaging where appropriate.

Integration fan-out

Röviden: Egy request vagy event után több downstream rendszer felé ágazik szét a munka.

Miért fontos: Ha mindent szinkronban csinálsz, a fő request latency-je és failure probability-je az összes downstreamtől függ.

Példa:

```
BookingCreated -> Payment, CRM, Email, Loyalty, Reporting, Partner update
```

Interview one-liner: Integration fan-out means one request or event triggers multiple downstream calls. I would keep the critical path synchronous and move non-critical notifications to async events or queues.

Coarse-grained service architecture

Röviden: Kevesebb, nagyobb service üzleti capability-k köré, nem túl sok apró microservice.

Miért fontos: Mule to Node.js migrációnál gyakran jobb kezdés, mert kevesebb network call, kevesebb distributed consistency probléma és egyszerűbb operáció.

Példa:

```
Coarse-grained: Booking Service, Payment Service, Customer Service
```

```
Fine-grained: BookingValidationService, BookingStatusService, BookingEmailTriggerService,  
BookingDatabaseService
```

Interview one-liner: I would start with coarse-grained services around clear business capabilities and split further only when independent scaling, deployment or ownership justifies it.

SOA vs ESB

Röviden: SOA architektúra-stílus. ESB egy technikai integration layer, amit gyakran SOA rendszerekben használnak.

Miért fontos: SOA nem egy termék. ESB lehet MuleSoft, WSO2, IBM App Connect, stb. SOA ESB nélkül is létezhet.

Példa:

SOA: Booking, Payment, Customer service-ek clear contractokkal
ESB: központi busz routinghoz, transformationhoz, protocol mediationhez

Interview one-liner: SOA is the architectural style, while an ESB is one possible integration backbone used to route, transform and mediate communication between services.

SOA és centralizáció

Röviden: SOA nem követeli meg kötelezően a central controller vagy ESB használatát, de enterprise SOA-ban gyakori a central governance és ESB.

Miért fontos: Interjún érdemes nem túl abszolútan fogalmazni. A centralizáció gyakori SOA implementation, de nem maga a SOA definíció.

Interview one-liner: SOA does not strictly require a centralized controller, but traditional enterprise SOA often uses central governance or an ESB for routing, transformation and orchestration.

ESB bottleneck vs API Gateway

Röviden: Az ESB gyakran több logikát tartalmaz, mint egy ideálisan vékony API Gateway, ezért könnyebben lesz technikai és csapatszintű bottleneck.

Miért fontos: Az ESB-ben gyakran routing, transformation, orchestration, connector logic és business workflow is van. API Gateway ideálisan inkább edge concernökre fókuszál: auth, routing, rate limit, TLS, logging.

- ESB bottleneck: runtime + business logic + transformation + csapat folyamat.
- API Gateway bottleneck: főleg edge/traffic bottleneck, ha vékony marad.
- Gateway is válhat mini-ESB-vé, ha túl sok business logic kerül bele.

Interview one-liner: An ESB is not always worse than a gateway, but it often accumulates more orchestration and transformation logic, so it can become both a runtime and organisational bottleneck.

Node.js alapú SOA / ESB

Röviden: Node.js-szel lehet SOA-szerű service-eket építeni, de Node.js önmagában nem klasszikus ESB product.

Miért fontos: Mule-ben sok integration capability beépített. Node.js-ben ezeket librarykkal és platform komponensekkel kell tudatosan standardizálni.

- Node.js integration service: igen.
- Node.js mint klasszikus ESB: nem igazán.
- Mule replacement Node-ban: possible, de kell routing, retries, validation, observability, queues, error handling standard

Interview one-liner: I would describe Node.js as a good technology for integration services, not as an ESB product by itself.

Choreography

Röviden: Nincs központi orchestrator. A service-ek eventekre reagálnak, majd saját eventeket publikálnak.

Miért fontos: Lazább couplingot ad és könnyű új consumert hozzáadni, de az end-to-end flow nehezebben látható. Kell tracing, correlation ID, idempotency és jó event contract.

Példa:

BookingCreated -> Payment Service reacts -> PaymentSucceeded -> Ticketing reacts -> TicketCreated -> Email reacts

Interview one-liner: Choreography means services coordinate a business process through events instead of a central orchestrator.

Aggregator

Röviden: Több forrásból érkező választ összegyűjt és egy egységes választ épít.

Miért fontos: Integration rendszereknél gyakori, például több partner API vagy internal service válaszát kell normalizálni és összerakni.

Példa:

```
Search API -> Partner A, Partner B, Partner C -> normalize -> sort -> combined result
```

Interview one-liner: An aggregator collects responses from multiple sources, normalizes them and returns one unified result.

Webhook

Röviden: HTTP-alapú event notification. Általában előre regisztrált callback URL-re küld a provider POST requestet, amikor event történik.

Miért fontos: Pub/sub-szerű, de nem message brokerrel, hanem HTTP endpointtal. Fontos a signature validation, idempotency, retry handling és replay protection.

- Registered webhook: előre beállított URL, sok eventet kapsz rá.
- Per-request callback URL: egy konkrét async művelethez adsz callback címet.
- Payloadban legyen eventId, eventType, business ID, timestamp, signature

Interview one-liner: A webhook is an HTTP-based event notification mechanism where the consumer registers a callback URL and the provider sends POST requests when subscribed events happen.

CRM

Röviden: Customer Relationship Management, ügyfélkapcsolat-kezelő rendszer.

Miért fontos: Integration role-ban a CRM gyakori downstream rendszer, amit booking, customer update vagy marketing preference változás után frissíteni kell.

Példa:

```
Booking created -> update customer history in CRM -> maybe update segment or loyalty-related data
```

Interview one-liner: CRM is a system for customer profiles, contact details, history, support, marketing or sales-related data.

Retention

Röviden: Megőrzési idő: a broker vagy event platform mennyi ideig tartja meg az üzeneteket/eventeket.

Miért fontos: Ha egy consumer leáll, retention alatt vissza tudja olvasni és be tudja hozni a lemaradást. Túl rövid retention adatvesztést okozhat.

Példa:

```
Retention: 7 days  
Consumer down: 2 hours  
Result: consumer can catch up later
```

Interview one-liner: Retention means how long a broker keeps messages or events before deleting them, which allows delayed consumers to catch up.

3. Resilience és failure handling

Graceful degradation

Röviden: Ha egy nem kritikus dependency vagy feature hibázik, a rendszer nem omlik össze teljesen, hanem csökkentett funkcionalitással tovább működik.

Miért fontos: A core flow megmaradhat akkor is, ha email, loyalty, recommendation vagy analytics ideiglenesen nem működik.

Példa:

```
Booking succeeds, but loyalty update is queued for later because the Loyalty service is down.
```

Interview one-liner: Graceful degradation means the system continues to provide the most important functionality even when a non-critical dependency fails.

Circuit breaker

Röviden: Ha egy downstream sokszor hibázik vagy timeoutol, egy ideig nem hívjuk tovább, hanem fail fast vagy fallback történik.

Miért fontos: Megakadályozza, hogy a rendszer újra és újra várjon egy beteg dependency-re, és ezzel saját magát is túlterhelje.

- Closed: normál működés.
- Open: nem hívjuk a dependency-t, gyors fallback/error.
- Half-open: próbahívás, hogy helyreállt-e.

Interview one-liner: A circuit breaker stops repeated calls to an unhealthy dependency and fails fast until it recovers.

Circuit breaker vs graceful degradation

Röviden: Circuit breaker technikai mechanizmus. Graceful degradation magasabb szintű üzleti viselkedés.

Miért fontos: Gyakran együtt használod őket: breaker kinyit, majd fallback vagy degraded mode aktiválódik.

Példa:

```
CRM slow -> breaker open -> do not call CRM for a while -> booking still succeeds -> CRM sync later
```

Interview one-liner: A circuit breaker is the resilience mechanism, while graceful degradation is the user-facing behavior that keeps the core flow working.

Bulkhead

Röviden: Erőforrás-elkülönítés: egy lassú vagy hibás dependency ne tudja elfogyasztani az egész rendszer connectionjeit, workereit vagy concurrency slotjait.

Miért fontos: Ha a CRM lassú, ne vigye magával a payment vagy booking core flow erőforrásait.

Példa:

```
Payment calls max 50 concurrent  
CRM calls max 10 concurrent  
Email calls max 20 concurrent
```

Interview one-liner: A bulkhead separates resources between dependencies or workloads so one failing downstream cannot consume all capacity.

Backpressure

Röviden: Amikor túl gyorsan jön a munka, a rendszer nem nyeli be végtelenül, hanem lassít, limitál, várakoztat vagy visszautasít.

Miért fontos: Védi a rendszert attól, hogy queue, memória, latency és timeout kontrollálatlanul nőjön.

- HTTP: 429 Too Many Requests vagy 503 Retry-After.

- Queue: max queue size, throttling, consumer scaling.
- Streams: pause/resume, drain event.

Interview one-liner: Backpressure means applying pressure back to the producer when the system receives more work than it can safely process.

Backpressure vs circuit breaker

Röviden: Backpressure a túl sok bejövő vagy feldolgozandó munkára reagál. Circuit breaker egy beteg downstream dependency-re reagál.

Miért fontos: Egy lassú downstream okozhat backpressure helyzetet is, de a két pattern nem ugyanaz.

Példa:

```
Incoming 1000 rps, capacity 300 rps -> backpressure  
CRM timeoutol -> circuit breaker
```

Interview one-liner: Backpressure controls overload, while a circuit breaker stops calls to an unhealthy dependency.

Per-downstream concurrency limit

Röviden: Dependency-nként külön limitálsz, hogy egyszerre hány párhuzamos hívás mehet felé.

Miért fontos: Ez a bulkhead gyakorlati implementációja. Megakadályozza, hogy egy lassú downstream minden outbound kapacitást elfoglaljon.

Példa:

```
Payment provider: max 50 concurrent calls  
CRM: max 10 concurrent calls  
External airline API: max 15 concurrent calls
```

Interview one-liner: A per-downstream concurrency limit limits how many calls my service can make to each dependency at the same time.

Stale data

Röviden: Elavult adat, ami régebbi állapotot mutat, mint a source of truth.

Miért fontos: Cache, read replica, async processing vagy event delay miatt előfordulhat. Nem mindig baj, de critical flow előtt újra kell validálni.

Példa:

```
Cache: flight price = 200 EUR  
Database/source of truth: flight price = 220 EUR
```

Interview one-liner: Stale data is data that is no longer fully up to date compared to the source of truth.

Locks és deadlocks

Röviden: A lock DB erőforrás ideiglenes lefoglalása consistency miatt. Deadlock akkor van, amikor két transaction körkörösén egymásra vár.

Miért fontos: API lassúság mögött lehet, hogy nem CPU vagy Node.js probléma van, hanem DB lock wait vagy deadlock.

Példa:

```
Tx A lockolja row 1-et, majd row 2-t akarja.  
Tx B lockolja row 2-t, majd row 1-et akarja.  
Mindkettő vár a másiktól: deadlock.
```

Interview one-liner: Locks are normal for consistency, but long lock waits or deadlocks can make an API look slow while the real bottleneck is database contention.

4. Data és consistency

ACID

Röviden: Adatbázis transaction garanciák: Atomicity, Consistency, Isolation, Durability.

Miért fontos: A local DB transaction akkor megbízható, ha ezek a garanciák érvényesülnek. Distributed rendszerben viszont nem mindig lehet egyetlen ACID transactionbe zárni mindent.

- Atomicity: minden vagy semmi.
- Consistency: érvényes állapotból érvényes állapotba.
- Isolation: párhuzamos transactionök ne zavarják össze egymást.
- Durability: commit után az adat megmarad.

Interview one-liner: ACID describes database transaction guarantees: all-or-nothing, valid state transitions, isolation between concurrent transactions, and durability after commit.

Local transaction vs distributed consistency

Röviden: Nem az app instance-ok száma dönti el, hanem hogy egy üzleti művelet hány független datastore/service boundary között ír adatot.

Miért fontos: Skálázott monolit több instance-szal is használhat egy közös DB-t és local transactiont. Distributed consistency akkor jön, ha több independent service/DB/external system állapotát kell összehangolni.

Példa:

```
Monolith instances -> same DB -> local transaction possible  
Booking Service DB + Payment Service DB + Ticketing external system -> distributed consistency problem
```

Interview one-liner: Scaling a monolith horizontally does not automatically create a distributed transaction problem. The problem appears when one business operation updates multiple independent systems or databases.

Saga

Röviden: Több local transactionból álló distributed workflow, ahol hiba esetén üzleti compensation lépések futnak.

Miért fontos: Microservice/distributed rendszerben nem jó default válasz egy nagy distributed transaction. Saga használható booking-payment-ticketing flow-khoz.

Példa:

```
Booking created -> Payment succeeded -> Ticket creation failed -> refund payment -> cancel booking
```

Interview one-liner: A saga manages a business process across services without one big distributed transaction, using local transactions and compensating actions.

Outbox pattern

Röviden: A business adatváltozást és az event rekordot ugyanabba a DB transactionbe mented, majd egy worker később kiküldi az eventet.

Miért fontos: Megoldja azt a klasszikus hibát, hogy a DB commit sikerül, de az event publish elveszik, vagy fordítva.

Példa:

```
BEGIN  
INSERT INTO bookings ...  
INSERT INTO outbox_events(type, payload, status) VALUES (...)  
COMMIT  
  
Worker reads PENDING outbox events and publishes them to the broker.
```

Interview one-liner: The outbox pattern stores the event in the same transaction as the business change, then a background worker publishes it reliably.

Reconciliation

Röviden: Utólagos egyeztetés több rendszer között, hogy ugyanazt az üzleti valóságot látják-e.

Miért fontos: Retries, outbox és sagák mellett is maradhatnak eltérések. Reconciliation job megtalálja és javítja vagy jelzi ezeket.

Példa:

```
Booking DB: booking 123 CONFIRMED
Payment provider: payment for booking 123 FAILED

Reconciliation detects the mismatch and triggers fix/manual review.
```

Interview one-liner: Reconciliation periodically compares state between systems and detects or fixes inconsistencies.

DB constraint

Röviden: Adatbázis által betartatott szabály, például NOT NULL, UNIQUE, PRIMARY KEY, FOREIGN KEY, CHECK.

Miért fontos: Az app validation jó user feedbackhez, de a DB constraint az utolsó védelmi vonal race condition, bugos script vagy rossz service esetén.

Példa:

```
email VARCHAR(255) NOT NULL UNIQUE
FOREIGN KEY (user_id) REFERENCES users(id)
CHECK (price >= 0)
```

Interview one-liner: A database constraint is a rule enforced by the database to protect data integrity, such as unique constraints, foreign keys and check constraints.

SQL query optimization

Röviden: Slow query okának keresése és javítása indexekkel, execution plannel, jobb joinokkal, paginationnel vagy query struktúrával.

Miért fontos: Production API lassúság gyakori oka a DB query latency, full table scan, rossz index, lock wait vagy N+1 query probléma.

- EXPLAIN / execution plan: hogyan futtatja a DB a queryt.
- Full table scan: sok vagy minden sort végignéz.
- Index: gyorsabb keresés/szűrés/sortolás bizonyos oszlopokon.
- N+1: sok kis query egy batch/join helyett.
- Deep OFFSET pagination: nagy offsetnél lassú lehet.

Interview one-liner: I would confirm the query is the bottleneck, inspect the execution plan, check indexes, joins, pagination, N+1 queries, pool usage and lock waits.

5. Security

Nonce

Röviden: Number used once: egyszer használatos véletlen/unique érték.

Miért fontos: Replay attack ellen használható, például signed webhooknál timestamp és signature mellett.

Példa:

```
X-Timestamp: 2026-04-28T12:00:00Z
X-Nonce: abc123random
X-Signature: ...

Ha ugyanaz a nonce már volt, reject.
```

Interview one-liner: A nonce is a unique value used only once, often with timestamps and signatures to prevent replay attacks.

Replay protection

Röviden: Megakadályozza, hogy egy régi, eredetileg érvényes requestet valaki újra beküldjön és újra elfogadtasson.

Miért fontos: Webhookoknál, payment eventeknél vagy signed requesteknél különösen fontos, mert egy régi request újrajátszása hibás üzleti állapotot okozhat.

- Signature validation: tényleg a provider küldte.
- Timestamp check: nem túl régi.
- Nonce vagy eventId tárolás: láttuk-e már.
- Idempotent processing: duplikátum nem okoz dupla hatást.

Interview one-liner: Replay protection prevents an old valid request from being reused later, usually with signatures, timestamps and nonces or event IDs.

Revocation

Röviden: Token, API key vagy permission visszavonása a normál lejárát előtt.

Miért fontos: JWT-knél érdekes, mert self-contained token esetén a token valid maradhat expiry-ig, ha nincs revocation stratégia.

- Short-lived access tokens.
- Refresh token revocation.
- Denylist/blacklist jti alapján high-risk esetekre.
- Opaque token/introspection, ha központi állapotellenőrzés kell.
- API key rotation és disable.

Interview one-liner: Revocation means invalidating a token, API key or permission before its normal expiry.

6. Observability

RED method

Röviden: Service/API szintű monitoring módszer: Rate, Errors, Duration.

Miért fontos: Megmutatja, mi történik a requestekkel: mennyi forgalom jön, mennyi hibázik, és mennyi ideig tartanak.

Példa:

```
Rate: 300 rps
Errors: 0.5% 5xx
Duration: p95 250 ms, p99 900 ms
```

Interview one-liner: RED monitors a service from the request perspective: rate, errors and duration.

USE method

Röviden: Resource szintű monitoring módszer: Utilization, Saturation, Errors.

Miért fontos: Segít megtalálni, hogy CPU, memória, DB pool, queue, network vagy event loop túlterhelt-e.

Példa:

```
Utilization: CPU 80%
Saturation: DB pool wait time high, queue length growing
Errors: connection errors
```

Interview one-liner: USE monitors resources: utilization, saturation and errors. I would use RED for symptoms and USE for underlying bottlenecks.

7. Gyors összefoglaló mondatok interjú előtt

Fogalom	Angol one-liner
p95/p99	Average latency can hide outliers, so I also check p95 and p99 tail latency.
Downstream	A downstream dependency is a system my service calls to complete its work.
Coupling	Loose coupling means systems communicate through clear contracts and can change internally without breaking each other.
Fan-out	I would keep the critical path synchronous and move non-critical fan-out to async events or queues.
SOA vs ESB	SOA is the architectural style, ESB is one possible integration backbone.
Backpressure	Backpressure means slowing or rejecting new work when the system cannot safely process more.
Bulkhead	Bulkheads isolate resources so one failing dependency cannot consume all capacity.
Saga	Saga uses local transactions and compensating actions instead of one distributed transaction.
Outbox	Outbox stores the event with the business change in the same DB transaction, then publishes it later.
Readiness	Readiness decides whether the instance can receive traffic; liveness decides whether it should be restarted.
Replay protection	Replay protection prevents old valid signed requests from being reused later.

Tipp: interjúban ne definíciókat sorolj. Mindig mondd hozzá, hogy productionben miért számít: latency, failure isolation, data consistency, observability, rollback/retry vagy consumer compatibility miatt.